

AI505  
Optimization

# **Discrete Optimization**

## **Local Search**

Marco Chiarandini

Department of Mathematics & Computer Science  
University of Southern Denmark

# Outline

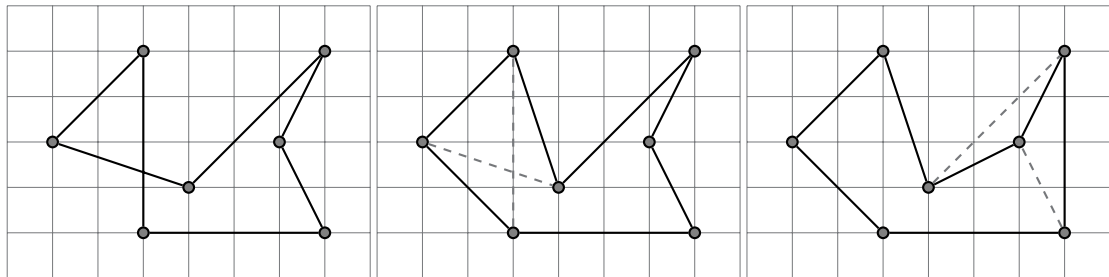
## 1. Local Search

# Outline

## 1. Local Search

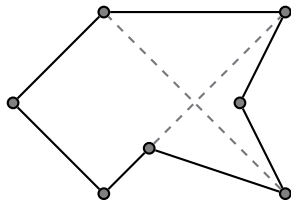
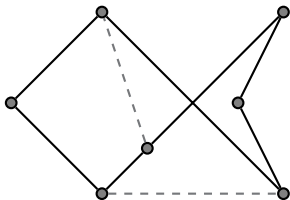
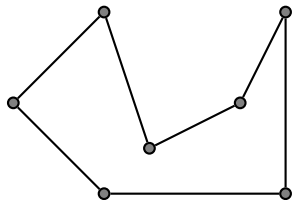
# ROHs: Reasoning, Local Search

Example on Traveling Salesman Problem:

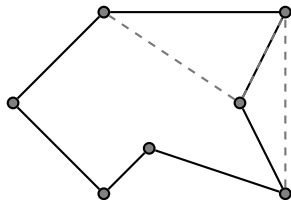
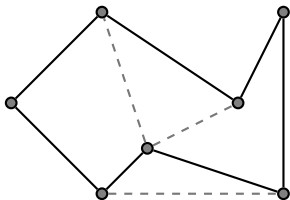
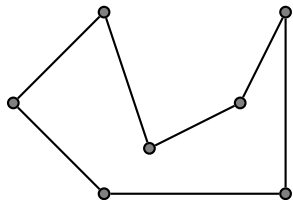


# ROHs: Reasoning, Metaheuristics

Accepting worsening changes



Trying different changes



# Local Search

- Model (Problem dependent)
  - Variables  $\rightsquigarrow$  solution representation, search space
  - Constraints:
    - implicit
    - one-way defining invariants
    - soft
  - Evaluation function
- Reasoning, Search (Problem independent)
  - Construction heuristics
  - Neighborhoods, Iterative Improvement, (Stochastic) local search
  - Metaheuristics: Tabu Search, Simulated Annealing, Iterated Local Search
  - Population based metaheuristics

# Local Search Algorithms

Given a (combinatorial) optimization problem  $\Pi$  and one of its instances  $\pi$ :

## 1. search space $S(\pi)$

- specified by the definition of (finite domain) **states** and their handling **implicit constraints**
- all together they determine the **representation of candidate solutions**
- common solution representations are discrete structures such as: sequences, permutations, partitions, graphs

Note: **solution set**  $S'(\pi) \subseteq S(\pi)$

# Local Search Algorithms (cntd)

## 2. **evaluation function** $f_{\pi} : S(\pi) \rightarrow \mathbf{R}$

- it handles the **soft constraints** and the objective function

## 3. **initialization function** $\text{init} : \emptyset \rightarrow S(\pi)$

(can be seen as a probability distribution  $\Pr(S(\pi) \times M(\pi))$  over initial search positions and memory states)



# Local Search Algorithms (cntd)

Further components [according to [HS]]

4. **neighborhood function**,  $N_\pi : S \rightarrow 2^{S(\pi)}$

- defines for each solution  $s \in S(\pi)$  a set of solutions  $N(s) \subseteq S(\pi)$  that are in some sense close to  $s$ .

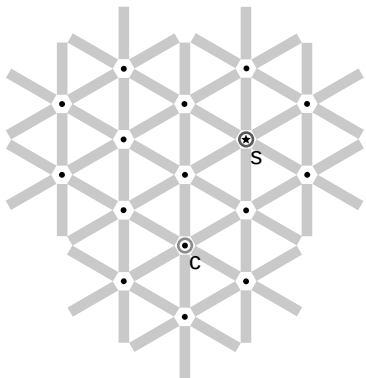
5. **transition model**  $\text{step} : S(\pi) \times M(\pi) \rightarrow S(\pi) \times M(\pi)$

(can be seen as a probability distribution  $\text{Pr}(S(\pi) \times M(\pi))$  over subsequent, neighboring search positions and memory states)

6. **termination predicate**  $\text{terminate} : S(\pi) \times M(\pi) \rightarrow \{\top, \perp\}$

(determines the termination state for each search position and memory state)

# Local search — global view



## Neighborhood graph

- vertices: candidate solutions (search positions)
- vertex labels: evaluation function
- edges: connect “neighboring” positions
- s: (optimal) solution
- c: current search position

# Neighborhood

## Definition (Neighborhood)

Neighborhood structure/type defined as a function  $N : S_\pi \rightarrow 2^S$  also defined as:  $\mathcal{N} : S \times S \rightarrow \{T, F\}$  or  $\mathcal{N} \subseteq S \times S$

- **neighborhood (set)** of candidate solution  $s$ :  $N(s) := \{s' \in S \mid \mathcal{N}(s, s')\}$
- **neighborhood size** is  $|N(s)|$
- neighborhood is **symmetric** if:  $s' \in N(s) \Rightarrow s \in N(s')$
- **neighborhood graph** of  $(S, N, \pi)$  is a directed graph:  $G_N := (V, A)$  with  $V = S$  and  $(uv) \in A \Leftrightarrow v \in N(u)$   
(if symmetric neighborhood  $\rightsquigarrow$  undirected graph)

# Local Search Algorithms

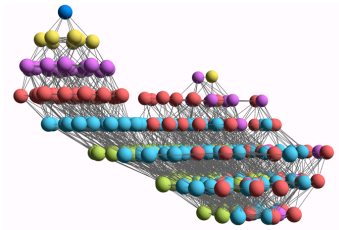
## Note:

- Local search implements a **walk** through the neighborhood graph
- Procedural versions of **init**, **step** and **terminate** implement sampling from respective probability distributions.
- Local search algorithms can be described as **Markov processes**:  
behavior in any **search state**  $\{s, m\}$  depends only  
on current position **s**  
higher order MP if (limited) memory **m**.

# Search Landscape

Given:

- Problem instance  $\pi$
- Search space  $S_\pi$
- Neighborhood function  $N : S \subseteq 2^S$
- Evaluation function  $f_\pi : S \rightarrow \mathbf{R}$



## Definition (Search Landscape)

The **search landscape**  $L$  is the **vertex-labeled neighborhood graph** given by the triplet  $\mathcal{L} = \langle S, N, f \rangle$ .

# Iterative Improvement

## Iterative Improvement (II):

determine initial candidate solution  $s$

**while**  $s$  has better neighbors **do**

└ choose a neighbor  $s'$  of  $s$  such that  $f(s') < f(s)$

└  $s := s'$

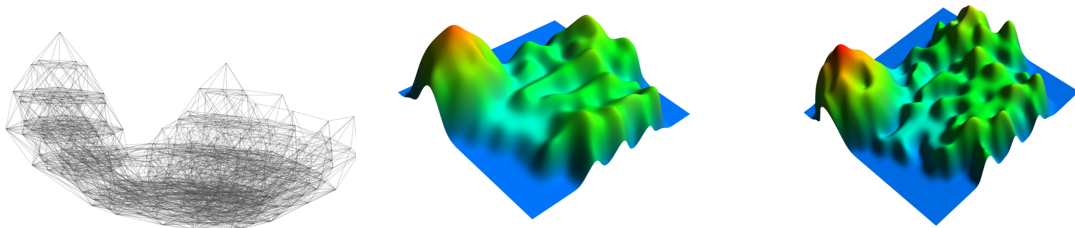
- If more than one neighbor has better cost then need to choose one (step, aka **heuristic pivot rule**):
  - first encountered
  - random among best ones
- The procedure ends in a **local optimum**  $\hat{s}$ :  
Def.: **Local optimum**  $\hat{s}$  w.r.t.  $N$  if  $f(\hat{s}) \leq f(s) \forall s \in N(\hat{s})$

# Definitions

## Definition:

- **Local minimum:** search position without improving neighbors wrt given evaluation function  $f$  and neighborhood function  $N$ ,  
i.e., position  $s \in S$  such that  $f(s) \leq f(s')$  for all  $s' \in N(s)$ .
- **Strict local minimum:** search position  $s \in S$  such that  $f(s) < f(s')$  for all  $s' \in N(s)$ .
- **Local maxima** and **strict local maxima:** defined analogously.

# Transition Graph



## Definition (Transition Graph of Iterative Improvement)

Given  $\mathcal{L} = \langle S, N, f \rangle$ , the transition graph of iterative improvement is a directed acyclic subgraph obtained from  $\mathcal{L}$  by deleting all arcs  $(i, j)$  for which it holds that the cost of solution  $j$  is worse than or equal to the cost of solution  $i$ .

It can be defined for other algorithms as well and it plays a central role in the theoretical analysis of proofs of convergence.

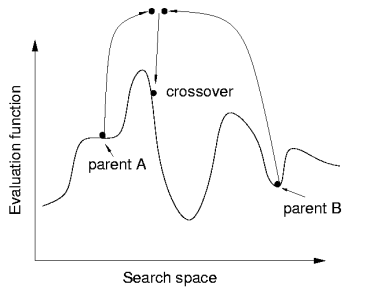
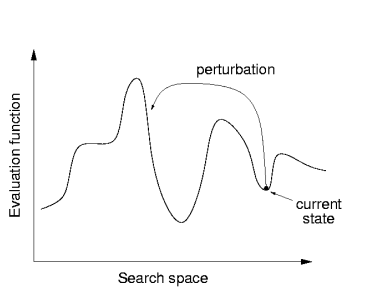
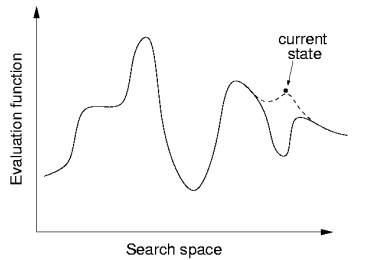
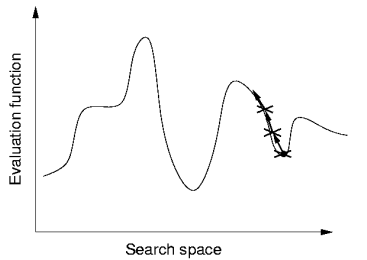
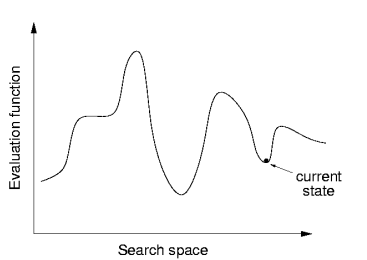


# Metaheuristics

How to avoid getting trapped in bad local optima?

- “Restart” + parallel search  
Avoid local optima  
Improve search space coverage
- use more complex neighborhood functions:
  - Variable Neighborhood Search and Large Scale Neighborhood Search  
**diversified** neighborhoods + incremental algorithmics  
("diversified"  $\equiv$  multiple, variable-size, and rich).
- allow non-improving moves
  - Tabu Search: Online learning of moves  
Discard undoing moves,  
Discard inefficient moves  
Improve efficient moves selection
  - Simulated annealing

# Idealized visualization of landscapes principles



# Evaluation function

## Evaluation (or cost) function:

- function  $f_{\pi} : S_{\pi} \rightarrow \mathbf{Q}$  that maps candidate solutions of a given problem instance  $\pi$  onto rational numbers (most often integer), such that global optima correspond to solutions of  $\pi$ ;
- used for assessing or ranking neighbors of current search position to provide guidance to search process.

## Evaluation vs objective functions:

- **Evaluation function**: part of LS algorithm.
- **Objective function**: integral part of optimization problem.
- Some LS methods use evaluation functions different from given objective function (e.g., guided local search).

# Constraint-based local search

From Van Hentenryck and Michel

If infeasible solutions are allowed, we count violations of constraints.

What is a violation?

Constraint specific:

- decomposition-based violations  
number of violated constraints, eg: alldiff
- variable-based violations  
min number of variables that must be changed to satisfy  $c$ .
- value-based violations  
for constraints on number of occurrences of values
- arithmetic violations
- combinations of these

# Constraint-based local search

From Van Hentenryck and Michel

Combinatorial constraints:

- $\text{alldiff}(x_1, \dots, x_n)$  (remember from CP):

Let  $a$  be an assignment with values  $V = \{a(x_1), \dots, a(x_n)\}$  and  $c_v = \#_a(v, x)$  be the number of occurrences of  $v$  in  $a$ .

Possible definitions for violations are:

- $\text{viol} = \sum_{v \in V} I(\max\{c_v - 1, 0\} > 0)$  value-based;  $I$  indicator function.
- $\text{viol} = \max_{v \in V} \max\{c_v - 1, 0\}$  value-based
- $\text{viol} = \sum_{v \in V} \max\{c_v - 1, 0\}$  value-based
- # variables with same value, variable-based, here leads to same definitions as previous three

Arithmetic constraints

- $l \leq r \rightsquigarrow \text{viol} = \max\{l - r, 0\}$
- $l = r \rightsquigarrow \text{viol} = |l - r|$
- $l \neq r \rightsquigarrow \text{viol} = 1$  if  $l = r$ , 0 otherwise

# Search space – Alternatives

## Search Space

Solution representations defined by the decision variables and the implicit constraints:

- permutations (implicit: alldifferent)
  - linear (scheduling problems)
  - circular (traveling salesman problem)
- arrays (implicit: assign exactly one; assignment problems: GCP)
- sets (implicit: disjoint sets, partition problems: graph partitioning, max indep. set)

~> Multiple **viewpoints** can be useful in local search!

# Search Space and Neighborhoods: A Formalization

A neighborhood (action type) is also defined by means of an operator  $\Delta$ .

An operator  $\Delta$  is a collection of operator functions  $\delta : S \rightarrow S$  (aka **move**) such that

$$s' \in N(s) \implies \exists \delta \in \Delta, \delta(s) = s'$$

## Definition

**$k$ -exchange neighborhood**: candidate solutions  $s, s'$  are neighbors iff  $s$  differs from  $s'$  in at most  $k$  solution components

## Examples:

- 2-exchange neighborhood for TSP  
(solution components = edges in given graph)

# Neighborhood Operator

Goal: providing a formal description of neighborhood functions for the three main solution representations:

- **Permutation**
  - **linear permutation**: Single Machine Total Weighted Tardiness Problem
  - **circular permutation**: Traveling Salesman Problem
- **Assignment**: SAT, CSP
- **Set, Partition**: Max Independent Set

A neighborhood function  $N : S \rightarrow 2^S$  is also defined through an operator.  
An **operator**  $\Delta$  is a collection of operator functions  $\delta : S \rightarrow S$  such that

$$s' \in N(s) \iff \exists \delta \in \Delta \mid \delta(s) = s'$$



## Digression: Permutations

$S_n$  indicates the set all permutations of the numbers  $\{1, 2, \dots, n\}$

$(1, 2, \dots, n)$  is the identity permutation  $\iota$ .

If  $\pi \in \Pi(n)$  and  $1 \leq i \leq n$  then:

- $\pi_i$  is the element at position  $i$
- $pos_\pi(i) = \pi_i^{-1}$  is the position of element  $i$

Alternatively, a permutation is a bijective function  $\pi(i) = \pi_i$

The permutation product  $\pi \cdot \pi'$  is the composition  $(\pi \cdot \pi')_i = \pi'(\pi(i))$

For each  $\pi$  there exists a permutation such that  $\pi^{-1} \cdot \pi = \iota$

$$\pi^{-1}(i) = pos_\pi(i)$$

$$\Delta_N \subset S_n$$

# Linear Permutations

**Swap** operator

$$\Delta_S = \{\delta_S^i \mid 1 \leq i \leq n\}$$

$$\delta_S^i(\pi_1 \dots \pi_i \pi_{i+1} \dots \pi_n) = (\pi_1 \dots \pi_{i+1} \pi_i \dots \pi_n)$$

**Interchange** operator

$$\Delta_X = \{\delta_X^{ij} \mid 1 \leq i < j \leq n\}$$

$$\delta_X^{ij}(\pi) = (\pi_1 \dots \pi_{i-1} \pi_j \pi_{i+1} \dots \pi_{j-1} \pi_i \pi_{j+1} \dots \pi_n)$$

( $\equiv$  set of all transpositions)

**Insert** operator

$$\Delta_I = \{\delta_I^{ij} \mid 1 \leq i \leq n, 1 \leq j \leq n, j \neq i\}$$

$$\delta_I^{ij}(\pi) = \begin{cases} (\pi_1 \dots \pi_{i-1} \pi_{i+1} \dots \pi_j \pi_i \pi_{j+1} \dots \pi_n) & i < j \\ (\pi_1 \dots \pi_j \pi_i \pi_{j+1} \dots \pi_{i-1} \pi_{i+1} \dots \pi_n) & i > j \end{cases}$$

# Circular Permutations

Reversal (**2-edge-exchange**)

$$\Delta_R = \{\delta_R^{ij} \mid 1 \leq i < j \leq n\}$$

$$\delta_R^{ij}(\pi) = (\pi_1 \dots \pi_{i-1} \pi_j \dots \pi_i \pi_{j+1} \dots \pi_n)$$

Block moves (**3-edge-exchange**)

$$\Delta_B = \{\delta_B^{ijk} \mid 1 \leq i < j < k \leq n\}$$

$$\delta_B^{ij}(\pi) = (\pi_1 \dots \pi_{i-1} \pi_j \dots \pi_k \pi_i \dots \pi_{j-1} \pi_{k+1} \dots \pi_n)$$

Short block move (**Or-edge-exchange**)

$$\Delta_{SB} = \{\delta_{SB}^{ij} \mid 1 \leq i < j \leq n\}$$

$$\delta_{SB}^{ij}(\pi) = (\pi_1 \dots \pi_{i-1} \pi_j \pi_{j+1} \pi_{j+2} \pi_i \dots \pi_{j-1} \pi_{j+3} \dots \pi_n)$$

# Assignments

An assignment can be represented as a mapping  $\sigma : \{X_1 \dots X_n\} \rightarrow \{v : v \in D, |D| = k\}$ :

$$\sigma = \{X_i = v_i, X_j = v_j, \dots\}$$

**One-exchange** operator

$$\Delta_{1E} = \{\delta_{1E}^{il} \mid 1 \leq i \leq n, 1 \leq l \leq k\}$$

$$\delta_{1E}^{il}(\sigma) = \{\sigma' : \sigma'(X_i) = v_l \text{ and } \sigma'(X_j) = \sigma(X_j) \ \forall j \neq i\}$$

**Two-exchange** operator

$$\Delta_{2E} = \{\delta_{2E}^{ij} \mid 1 \leq i < j \leq n\}$$

$$\delta_{2E}^{ij}(\sigma) = \{\sigma' : \sigma'(X_i) = \sigma(X_j), \sigma'(X_j) = \sigma(X_i) \text{ and } \sigma'(X_l) = \sigma(X_l) \ \forall l \neq i, j\}$$

# Partitioning

An assignment can be represented as a partition of objects selected and not selected

$s : \{X\} \rightarrow \{C, \bar{C}\}$  (it can also be represented by a bit string)

**One-addition** operator

$$\Delta_{1E} = \{\delta_{1E}^v \mid v \in \bar{C}\}$$

$$\delta_{1E}^v(s) = \{s : C' = C \cup v \text{ and } \bar{C}' = \bar{C} \setminus v\}$$

**One-deletion** operator

$$\Delta_{1E} = \{\delta_{1E}^v \mid v \in C\}$$

$$\delta_{1E}^v(s) = \{s : C' = C \setminus v \text{ and } \bar{C}' = \bar{C} \cup v\}$$

**Swap** operator

$$\Delta_{1E} = \{\delta_{1E}^v \mid v \in C, u \in \bar{C}\}$$

$$\delta_{1E}^v(s) = \{s : C' = C \cup u \setminus v \text{ and } \bar{C}' = \bar{C} \cup v \setminus u\}$$

# Fundamental Properties

The behavior and performance of an LS algorithm on a given problem instance crucially depends on properties of the respective search landscape.

## Simple properties:

- **search space size**  $|S|$
- **reachability**: solution  $j$  is reachable from solution  $i$  if neighborhood graph has a path from  $i$  to  $j$ .
  - **strongly connected neighborhood graph**
  - **weakly optimally connected neighborhood graph**
- distance between solutions
- neighborhood size (ie, degree of vertices in neigh. graph)
- cost of fully examining the neighborhood
- relation between different neighborhood functions  
(if  $N_1(s) \subseteq N_2(s)$  for all  $s \in S$  then  $N_2$  dominates  $N_1$ )

# Neighborhood Domination

## Definition (Neighborhood Domination)

Let  $N_1$  and  $N_2$  be two different neighborhood functions for the same instance  $(S, f, \pi)$  of a combinatorial optimization problem.

If for all solutions  $s \in S$  we have  $N_1(s) \subseteq N_2(s)$  then we say that  $N_2$  **dominates**  $N_1$

## Example:

In TSP, 1-insert is dominated by 3-exchange.

(1-insert corresponds to 3-exchange and there are 3-exchanges that are not 1-insert)

# Computational Complexity of LS

For a local search algorithm to be effective, search initialization and individual search steps should be efficiently computable.

**Complexity class  $\mathcal{PLS}$ :** class of problems for which a local search algorithm exists with polynomial time complexity for:

- search initialization
- any single search step, including computation of evaluation function value

For any problem in  $\mathcal{PLS}$  ...

- local optimality can be verified in polynomial time
- improving search steps can be computed in polynomial time
- **but:** finding local optima may require super-polynomial time



# Computational Complexity of LS

**$\mathcal{PLS}$ -complete**: Among the most difficult problems in  $\mathcal{PLS}$ ;  
if for any of these problems local optima can be found  
in polynomial time, the same would hold for all problems in  $\mathcal{PLS}$ .

## Some complexity results:

- TSP with  $k$ -exchange neighborhood with  $k > 3$  is  $\mathcal{PLS}$ -complete.
- TSP with 2- or 3-exchange neighborhood is in  $\mathcal{PLS}$ , but  $\mathcal{PLS}$ -completeness is unknown.

# Examples: Search Space

The **search space** is the set of all candidate solutions that may be visited during the solving process

- Only feasible solutions are considered in local search  
The objective function is defined on the whole search space
- The internal solution structure may not be specified (indirect solution representation)
- Solutions are points in the search space
- Initial feasible solution(s) may be:
  - Generated at random
  - Obtained by constructive search
  - Obtained heuristically in other ways

# Examples: Search Space and Initializations

How to generate solutions guaranteed to be picked **uniformly at random** for the following problems?

- Travelling salesman  
Generate a random permutation of the  $N$  cities
- Binary Knapsack  
Generate a random binary vector of length equal to the number of available items and reject sets of items that do not meet the capacity constraint(s)  
Possibly (very) inefficient if the problem is tightly constrained
- Clique partitioning problem / community detection problem
  1. Draw a number of bins at random according to a specific distribution (see [https://doi.org/10.1016/0097-3165\(83\)90009-2](https://doi.org/10.1016/0097-3165(83)90009-2))
  2. Randomly assign each vertex to one of those bins (some bins may end up empty)
- Graph Coloring
- Scheduling

# Examples: Neighbourhoods

What are similar solutions?

- “Parts” of the two solutions are somehow identical
- Similar performance (in most cases)
- Connect the whole space

Symmetric Travelling Salesman Problem:

- Tour length is the sum of the lengths of the tour edges
- Solutions are similar if they differ in a small number of edges
- 2-opt and 3-opt moves

Asymmetric Travelling Salesman Problem

- Tour length is the sum of the lengths of the tour arcs
- Solutions are similar if they differ in a small number of arcs
- Due to asymmetry, 2-opt moves and some 3-opt moves are not suitable because they reverse parts of the tour
- Insertion move as a special case of 3-opt

# Neighbourhood Exploration

Local search algorithms in a nutshell

1. Visit neighbours of a current solution
2. Decide whether to reject them or to accept one as the next solution
3. Repeat

Local move generation

- Enumeration
- Random sampling with replacement
- Random sampling without replacement

# Neighbourhood exploration

## Local move enumeration

- Full neighbourhood exploration
- Enumeration order dictated by convenience
- Filter out invalid moves if needed
- Neighbourhood size may not be known in advance

## Random sampling with replacement

- Sampling uniformly at random typically preferred
- Rejection sampling useful when neighbourhood size not known in advance

# Neighbourhood exploration

## Random sampling without replacement

- Partial to full neighbourhood exploration
- Random enumeration order to avoid search bias
- Filter out invalid moves if needed (rejection)
- Neighbourhood size may not be known in advance
- Generating (pseudo-)random permutations
  - Fisher-Yates shuffle
  - Linear congruential generator (low quality but constant space)
  - A plethora of other approaches

# Digression: Random Numbers

Needed to randomize search (eg, initial solution, neighborhood exploration)

Carachtersitics of a good pseudo-random generator  
(from stochastic simulation)

- long period
- uniform unbiased distribution
- uncorrelated (time series analysis)
- efficient

Suggested: MRG32k3a by L'Ecuyer <http://www.iro.umontreal.ca/~lecuyer/>



# Ideal Random Shuffle – Fisher-Yates shuffle

Let's consider a sequence of  $n$  elements:  $\{e_1, e_2, \dots, e_n\}$ .

The **ideal random shuffle** is a permutation chosen uniformly at random from the set of all possible  $n!$  permutations.

- $\pi_1$  is uniformly randomly chosen among  $\{e_1, e_2, \dots, e_n\}$ .
- $\pi_2$  is uniformly randomly chosen among  $\{e_1, e_2, \dots, e_n\} - \{\pi_1\}$ .
- $\pi_3$  is uniformly randomly chosen among  $\{e_1, e_2, \dots, e_n\} - \{\pi_1, \pi_2\}$
- ...

Joint probability of  $(\pi_1, \pi_2 \dots \pi_n)$  is  $\frac{1}{n} \cdot \frac{1}{n-1} \cdot \dots \cdot 1 = \frac{1}{n!}$

```
long int* Random::generate_random_array(const int& size) { // C code
    long int  i, j, help;
    long int  *v = new long int[size];
    for ( i = 0 ; i < size; i++ )
        v[i] = i;
    for ( i = 0 ; i < size-1 ; i++) {
        j = (long int) ( ranU01( ) * (size - i));
        help = v[i];
        v[i] = v[i+j];
        v[i+j] = help;
    }
    return v; }
```

# Neighbourhood exploration

## Random sampling without replacement

It can be implemented using a Fisher-Yates shuffled mask.

Python implementation:

```
def fisher_yates_iter(n):  
    p = list(range(n))  
    for i in range(n-1, -1, -1):  
        r = random.randrange(i+1)  
        yield p[r]  
        # p[r], p[i] = p[i], p[r] # preserve the permutation  
        p[r] = p[i] # lazy
```

# Neighbourhood exploration

## Random sampling without replacement

It can be implemented using a Fisher-Yates shuffled mask.

Python implementation:

```
def sparse_fisher_yates_iter(n):  
    p = dict()  
    for i in range(n-1, -1, -1):  
        r = random.randrange(i+1)  
        yield p.get(r, r)  
        if i != r:  
            # p[r] = p.pop(i, i) # saves memory, takes time  
            p[r] = p.get(i, i) # lazy, but faster
```

# Reservoir Sampling

How to select an element at random from a sequence of  $n$  elements where  $n$  is unknown?

- Keep the first item in memory.
- When the  $i$ -th item arrives (for  $i > 1$ ):
  - with probability  $1/i$ , keep the new item (discard the old one)
  - with probability  $1 - 1/i$  keep the old item (ignore the new one)

Probability of an element to be selected:

$$\Pr(\text{being selected once}) \cdot \Pr(\text{not being swapped later}) = 1 \cdot \prod_{i=2}^n \left(1 - \frac{1}{i}\right)$$

Extension:

How to select a sample of  $k$  elements from a sequence  $S$  of unknown length?

- Keep the first  $k$  items in memory.
- When the  $i$ th item arrives (for  $i > k$ ):
  - with probability  $k/i$ , keep the new item (discard an old one, selecting which to replace at random, each with chance  $1/k$ )
  - with probability  $1 - k/i$ , keep the old items (ignore the new one)

For the first element of the sequence the probability of being selected is:

$$\Pr(\text{being selected first time}) \cdot \prod_{i=k+1}^{|S|} \Pr(\text{not being swapped}) = 1 \cdot \prod_{i=k+1}^{|S|} \left(1 - \frac{k}{i} \cdot \frac{1}{k}\right)$$

where

$$\Pr(\text{not being swapped}) = 1 - \Pr(\text{swapping exactly that one}) = 1 - \frac{k}{i} \cdot \frac{1}{k}$$

$$\Pr(\text{not being swapped}) = \Pr(\text{not swapping}) + p(\text{swapping one of the others}) = \left(1 - \frac{k}{i}\right) + \left(\frac{k}{i} \cdot \frac{k-1}{k}\right)$$

## Implementation:

```
// S has items to sample, R will contain the result
ReservoirSample(S[1..n], R[1..k])
  // fill the reservoir array
  for i = 1 to k
    R[i] := S[i]

  // replace elements with gradually decreasing probability
  for i = k+1 to n
    j := random(1, i)  // important: inclusive range
    if j <= k
      R[j] := S[i]
```